

VeriBeca

Guía de estudio del proyecto

Asignación transparente de becas con DSL tipado, IA y Blockchain

Teoría de Computación 2 · Universidad Champagnat

1. ¿Qué es VeriBeca? (en una frase)

VeriBeca hace que la asignación de becas sea justa y auditable: la institución escribe sus criterios en un lenguaje simple, el sistema los aplica igual para todos, una IA ordena por necesidad, y cada decisión queda sellada en blockchain para que cualquiera pueda verificarla.

El problema que resuelve

Hoy, cuando una universidad o municipio reparte becas, nadie de afuera puede comprobar por qué a uno le dieron y a otro no. Los criterios son confusos, la evaluación es manual y no queda evidencia inalterable. Eso genera desconfianza y sospechas de favoritismo.

La solución, paso a paso

- **1. Escribir la regla** en un lenguaje propio y legible (el DSL).
- **2. Validar:** un "type-checker" revisa que la regla esté bien formada antes de aplicarla.
- **3. Aplicar:** el intérprete evalúa la regla sobre cada postulante (elegible / no elegible).
- **4. Priorizar:** una IA ordena a los elegibles por vulnerabilidad.
- **5. Sellar:** por cada decisión se calcula un hash y se registra en blockchain.

2. El lenguaje de reglas (DSL): guía completa

Esta es la pieza central del proyecto. Permite escribir los criterios de elegibilidad en un lenguaje propio, simple y verificable.

Estructura básica

```
SI <condición> ENTONCES elegible
```

- **SI** → arranca la regla (obligatorio).
- **<condición>** → lo que tiene que cumplirse para ser elegible.
- **ENTONCES elegible** → el cierre (obligatorio).

Si el postulante cumple la condición es **elegible**; si no, **no elegible**.

Variables disponibles (los datos de cada postulante)

Variable	Tipo	Qué representa
ingreso_familiar	número	Ingreso de la familia (ej: 250000)
promedio	número	Promedio académico (ej: 7.5)
integrantes_familia	número	Cuántas personas viven en el hogar
distancia_km	número	Distancia al lugar de estudio
trabaja	booleano	Si trabaja o no (verdadero / falso)

carrera	texto	Nombre de la carrera (ej: "medicina")
---------	-------	---------------------------------------

Operadores de comparación

Operador	Significa	Ejemplo
<	menor que	ingreso_familiar < 300000
<=	menor o igual	promedio <= 6
>	mayor que	distancia_km > 50
>=	mayor o igual	promedio >= 7
==	igual a	carrera == "medicina"
!=	distinto de	carrera != "derecho"

Regla de tipos: los operadores <, <=, >, >= SOLO funcionan con números. Con texto y booleanos solo se puede usar == y !=. Por eso 'carrera < 5' es inválido y el sistema lo rechaza.

Conectores (para combinar condiciones)

Conector	Significa	Ejemplo
Y	ambas deben cumplirse	ingreso_familiar < 300000 Y promedio >= 7
O	al menos una debe cumplirse	distancia_km > 50 O ingreso_familiar < 150000
NO	niega una condición	NO trabaja == verdadero

Cómo escribir los valores

- **Números:** sin comillas → 300000, 7, 8.5
- **Texto:** siempre entre comillas dobles → "medicina", "ingenieria"
- **Booleanos:** las palabras verdadero o falso → trabaja == verdadero

Carreras posibles en el dataset: "medicina", "ingenieria", "derecho", "psicologia", "contaduria".

Paréntesis (para agrupar)

Cuando mezclas Y con O conviene agrupar con paréntesis:

SI (ingreso_familiar < 200000 O distancia_km > 50) Y promedio >= 6 ENTONCES elegible

Significa: buen promedio (>=6) **y además** (bajos ingresos **o** vivir lejos). Sin paréntesis, el Y "agarra más fuerte" que el O (como la multiplicación antes que la suma).

Ejemplos completos explicados

SI ingreso_familiar < 300000 Y promedio >= 7 ENTONCES elegible

Elegible si gana menos de 300.000 y tiene 7 o más de promedio.

SI carrera == "medicina" Y promedio >= 7 ENTONCES elegible

Elegible si estudia medicina y tiene buen promedio.

SI NO trabaja == verdadero Y ingreso_familiar < 250000 ENTONCES elegible

Elegible si **no** trabaja y tiene ingresos bajos.

SI promedio >= 8 0 integrantes_familia >= 5 ENTONCES elegible

Elegible si tiene excelente promedio o si es familia numerosa (5 o más).

3. Cómo funciona por dentro (el "mini-compilador")

Cuando escribís una regla, el sistema la procesa en 4 etapas, igual que un lenguaje de programación real. Esto cubre las unidades de la materia:

- **Lexer:** parte el texto en "piezas" (tokens): palabras, números, operadores.
- **Parser:** arma con esas piezas un árbol (AST) que representa la estructura de la regla.
- **Type-checker:** revisa que los tipos sean correctos (no comparar texto con número, que la variable exista, etc.).
- **Intérprete:** recorre el árbol y evalúa la regla con los datos de cada postulante.

Gracias al type-checker, una regla mal escrita se detecta **antes** de aplicarla, evitando errores que perjudiquen a una persona real.

4. Cómo usamos la Inteligencia Artificial

Qué hace

La IA **no decide quién es elegible** (eso lo hace la regla). La IA **ordena por prioridad** a los que ya son elegibles, según su vulnerabilidad socioeconómica. Así, si hay más postulantes que becas, se atiende primero a quien más lo necesita.

Qué herramienta y por qué

- **Herramienta:** scikit-learn (la librería de machine learning más usada en Python).
- **Por qué:** simplicidad e **interpretabilidad**. En decisiones sensibles es más importante poder explicar por qué alguien tiene más prioridad que usar un modelo "caja negra".
- **Qué aporta:** un orden de prioridad consistente y explicable, igual para todos.

Cómo calcula la prioridad (el score)

Combina tres indicadores normalizados en un puntaje de 0 a 1 (a mayor puntaje, mayor vulnerabilidad → más prioridad):

- **Ingreso familiar (peso 50%):** a menor ingreso, mayor prioridad.
- **Integrantes de la familia (peso 30%):** a más integrantes, mayor prioridad.
- **Distancia (peso 20%):** a mayor distancia, mayor prioridad.

Para la memoria técnica también se demuestra el uso de scikit-learn entrenando una regresión lineal que reproduce el score ($R^2 > 0.99$).

5. Cómo usamos la Blockchain

Para qué sirve (no es "de moda")

La blockchain se usa para **una cosa concreta**: que una decisión, una vez tomada, **no se pueda alterar** y cualquiera pueda verificarla. Eso es imposible de garantizar con una planilla o una base de datos interna, donde alguien podría "retocar" un resultado.

Cómo funciona, paso a paso

- Por cada decisión se calcula un **hash** (un código único) de: los datos del postulante + la regla aplicada + el resultado.
- Ese hash se envía a un **smart contract** (un programa que vive en la blockchain) llamado **AuditoriaBecas**.
- El contrato guarda el hash con la fecha/hora. Queda **sellado e inalterable**.
- Cualquiera puede entrar al explorer público y verificar que esa decisión existió y cuándo.

Conceptos clave (para el pitch)

Término	Qué es, en simple
Testnet	Una versión de prueba de la blockchain, con dinero de mentira (gratis). Usamos Ethereum Sepolia.
Wallet	Una cuenta para firmar operaciones. Tiene dirección pública y clave privada (secreta).
Smart contract	Un programa que vive en la blockchain. El nuestro guarda los hashes de las decisiones.
Hash	Una "huella digital" única de la decisión. Si algo cambia, el hash cambia.
Gas	La comisión de cada operación. En testnet se paga con monedas gratis.

Evidencia real (ya funcionando)

- Contrato desplegado en Ethereum Sepolia (verificable en sepolia.etherscan.io).
- Una transacción real de auditoría registrada y verificable.
- Si la red falla, el sistema usa un "fallback" local: registra el mismo hash en un archivo, así la demo nunca se interrumpe.

6. Cómo cubre la consigna de la materia

Tema de la materia	Cómo se aplica en VeriBeca
Lenguajes de programación	DSL propio de reglas de elegibilidad
Sistemas de tipos	Type-checker que valida las reglas antes de aplicarlas

Diseño de compiladores	Pipeline lexer → parser → AST → intérprete
Seguridad / Blockchain	Hash de cada decisión + smart contract en testnet
Inteligencia Artificial	Scoring de vulnerabilidad con scikit-learn

Frase para el pitch: "Integramos las 4 unidades de la materia + IA + Blockchain en un solo flujo real, no como piezas separadas."

7. Cómo levantar el sistema

Son dos programas que corren a la vez. En la carpeta del proyecto, un comando lo levanta todo y abre el navegador en `http://localhost:8501`:

```
cd "d:\Claude\Proyectos\Hackaton"
.venv\Scripts\python.exe -m uvicorn backend.main:app    (Terminal 1, el backend)
.venv\Scripts\python.exe -m streamlit run frontend/app.py (Terminal 2, la app)
```

En la app: escribir una regla → Validar → subir `data\postulantes.csv` → Cargar → Evaluar.

8. Datos clave para el pitch

- MVP funcionando de punta a punta, con **27 pruebas automáticas** en verde.
- Sobre 100 postulantes de prueba: evaluó y priorizó **24 elegibles** en segundos.
- **Contrato real** desplegado y **transacción verificable** en Etherscan (Sepolia).
- Robusto: si se cae internet, sigue funcionando en modo de respaldo.
- Frase de cierre: "De 'confíen en nosotros' a 'verifíquenlo ustedes mismos'."

9. Posibles preguntas del jurado (con respuestas)

¿Para qué blockchain? ¿No es moda?

No la usamos de adorno. La usamos para que una decisión, una vez tomada, no se pueda cambiar y cualquiera la pueda auditar. Eso no lo garantiza una planilla ni una base de datos interna.

¿La IA decide quién recibe la beca?

No. La elegibilidad la define la regla (transparente). La IA solo ordena por prioridad a los ya elegibles, con un puntaje explicable. Nunca es una "caja negra" que decide sola.

¿Qué pasa si la regla está mal escrita?

El type-checker la rechaza antes de aplicarla y avisa el error, así no perjudica a nadie.

¿Qué funciona hoy y qué quedó pendiente?

Funciona todo el flujo: escribir/validar regla, evaluar, priorizar y generar el certificado (on-chain real o en respaldo). Pendiente para el futuro: login de usuarios, multi-institución, base de datos y despliegue productivo.

¿Cómo se sostiene en el tiempo? ¿Quién pagaría?

Modelo SaaS por institución (suscripción) más servicios de implementación. Pagan las instituciones que asignan las ayudas, que ganan eficiencia y reputación.